

Detect SQL Injection Vulnerabilities Using Various SQL Injection Detection Tools

Execution mode	Authorized, isolated training lab; no public systems were targeted.
Output status	Representative/sanitized lab output recorded for submission. Replace with instructor-required screenshots if a live lab run is performed.

Aim

Confirm SQL injection indicators with sqlmap detection mode and OWASP ZAP without dumping application data.

Theory

Detection focuses on proving that a parameter changes query behavior; exploitation goes further by extracting data. This report stops at confirmation and uses the DVWA-style lab target only.

Tools and Lab Scope

- sqlmap detection mode
- OWASP ZAP Active Scan
- DVWA SQLi module at low security

Procedure and Recorded Execution

Step 1. Prepare an authenticated lab request - Use the disposable session cookie and the authorized URL.

target: `http://192.168.56.30/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit`
cookie: `PHPSESSID=REDACTED; security=low`

Step 2. Run sqlmap detection - Use moderate level/risk and do not request databases or tables.

`sqlmap ... --batch --level=3 --risk=2 --forms`
parameter id appears injectable

Step 3. Classify the technique - Record only the detection method reported by the tool.

technique: `boolean-based blind`
DBMS fingerprint: `MySQL-compatible training backend`

Step 4. Cross-check with ZAP - Run the SQLi rule against the same URL and compare evidence.

ZAP alert: `SQL Injection, parameter id`
status: `corroborated`

Step 5. Close the test - Export the confirmation report and stop without extraction.

SQLi detection output

Both tools identified the id parameter as injectable. The exercise stopped at detection, with no database enumeration or data extraction performed.

Tool	Finding	Scope
sqlmap	id injectable; boolean-based blind; DBMS fingerprinted	Detection only
OWASP ZAP	SQL Injection alert on id	Detection only
Data access	No databases/tables dumped	Confirmed
Recommended fix	Parameterized queries and input validation	Retest after patch

Result

SQL injection detection completed and was corroborated by two tools. The distinction between finding a flaw and extracting data was maintained throughout the lab.

Safety and Submission Notes

- Do not include session cookies in a report.
- Use detection-only flags when data extraction is not required.
- Confirm findings manually and retest after remediation.

Viva Questions and Answers

Detection vs exploitation?

Detection proves a parameter is vulnerable; exploitation uses that flaw to access data or perform actions.

How does time-based blind SQLi work?

A conditional delay changes response time when the injected condition is true.

What do --level and --risk control?

They control the breadth and aggressiveness of sqlmap tests.

How can a WAF cause a false negative?

It may block or normalize test payloads before they reach the vulnerable code.

Why use two tools?

Independent evidence can reduce missed findings and tool-specific errors.